FORMATION REACT.JS · DÉMO 004-F

# React Compiler : les changements

On part de l'étape 4 (`etape-4-usecallback-usememo`), où on a optimisé à la main avec `React.memo`, `useMemo` et `useCallback`. Avec le React Compiler, on enlève tout ça : c'est lui qui fait ce travail. Le projet prêt : `react-compiler`.

## 1. C'est quoi le React Compiler ?

- C'est un outil de React qui fait les optimisations à notre place.
- Avant, on ajoutait nous-mêmes `React.memo`, `useMemo` et `useCallback`. Maintenant, on écrit notre code normal, et c'est lui qui s'en occupe au moment du build.
- Il est stable : version **1.0** depuis octobre 2025. Il marche avec React 19.

## 2. Installer

```
npm install -D babel-plugin-react-compiler
```

## 3. Configurer Vite (`vite.config.js`)

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [
    react({
      // React Compiler : il fait les optimisations à notre place
      babel: { plugins: ["babel-plugin-react-compiler"] },
    }),
  ],
});
```

## 4. Enlever React.memo, useMemo et useCallback ( `src/components/Cars.jsx` )

```
// Avant (étape 4)
const MemoizedCar = React.memo(Car);

const onSelect = useCallback((make) => {
  console.log("Selected:", make);
}, []);

const carStyle = useMemo(
  () => ({ padding: "10px", marginBottom: "10px", border: "1px solid #ccc", cursor: "pointer"
}),
  []
);

// Après : un composant, une fonction et un objet normaux
<Car … />

const onSelect = (make) => {
  console.log("Selected:", make);
};

const carStyle = {
  padding: "10px",
  marginBottom: "10px",
  border: "1px solid #ccc",
  cursor: "pointer",
};
```

## 5. Le bouton See More : `(v) => !v`

```
// ❌ Mauvais exemple (en commentaire dans le projet)
onClick={() => setSeeMore(!seeMore)}

// ✅ Bon
onClick={() => setSeeMore((v) => !v)}
```

- Avec `!seeMore`, la fonction du bouton dépend de `seeMore` : le compilateur la recrée à chaque clic.
- Avec `(v) => !v`, elle ne dépend de rien : elle reste en cache.
- C'est aussi la bonne pratique officielle de React quand le nouveau state dépend de l'ancien (la fonction de mise à jour).
- Dans notre démo, ça change aussi les re-render des Car : avec `!seeMore`, « Car BMW Rendered » revient au clic sur See More ; avec `(v) => !v`, non. On le vérifie avec le `console.log` dans Car.

## 6. Tester

```
cd react-compiler
npm install
npm run dev
```

Console ouverte, on clique sur « Set Make to BMW » puis sur « See More » :

- « Set Make to BMW » : « Car BMW Rendered ». Normal, la liste de voitures a changé.
- « See More » : rien. Le compilateur fait le travail, sans `React.memo`, `useMemo` ni `useCallback`.

- Avec le mauvais exemple (`!seeMore`) : « Car BMW Rendered » revient au clic sur See More.

## 7. Vérifier

- **ESLint** : `npm run lint`. Le plugin des hooks (déjà dans un projet Vite) contient aussi les règles du compilateur.
- **Le healthcheck** : il dit combien de composants sont compilés.
- **React DevTools** : le badge « Memo ✨ » sur les composants optimisés, et le Profiler pour voir les re-render.

```
npm run lint
npx react-compiler-healthcheck@latest
```

## Avantages et inconvénients

- **Avantages** : moins de code (plus de `React.memo`, `useMemo`, `useCallback` partout), pas d'oubli, pas de tableau de dépendances à gérer.
- **Inconvénients** : il faut respecter les règles de React, un peu plus de temps de build, et on vérifie toujours le résultat (console, DevTools).

## Documentation

- React Compiler : https://react.dev/learn/react-compiler
- `memo` et le React Compiler : https://react.dev/reference/react/memo
- La fonction de mise à jour du state : https://react.dev/reference/react/useState#updating-state-based-on-the-previous-state
- La version 1.0 : https://react.dev/blog/2025/10/07/react-compiler-1